

PRACTICUM REPORT: THE AMDAHL REALITY GAP

Empirical Speedup Fitting, Micro-Architectural Bottlenecks, and Loop Scheduling in OpenMP

Field	Value
Student Full Name:	Kemelov Nurbolat
Institutional Email / Git URL:	https://github.com/Nurbolat-yes/314-labs
Student ID Number	230103027
Practicum Date	25 September 2026
Operating System	Windows
CPU Model & Generation	11th Gen Intel(R) Core(TM) i5-1155G7 @ 2.50GHz
Physical Cores	4
Logical SMT Threads	8
Target Workload Size N	13,027,000
Cache Line Size Used for Analysis	64 bytes

1. Workload Formulation and Hardware Probe

The workload size was derived from the last four digits of Student ID 230103027. The last four digits are 3027.

$$N = 10,000,000 + (3027 \times 1,000) = 13,027,000$$

The hardware probe reported an 11th Gen Intel Core i5-1155G7 processor with 4 physical cores and 8 logical processors. The raw PowerShell output was saved in hw_info.txt.

2. Computational Kernel

For every integer i in the interval $[1, N]$, the program calculated the Collatz stopping time. It tracked the maximum stopping time and accumulated the sum of all stopping times modulo 1,000,000,007 as a verification checksum.

Verification Item	Result
N	13,027,000
Maximum stopping time	688
Checksum (mod 1,000,000,007)	57,998,076

3. Sequential Baseline

The sequential implementation was compiled with GCC optimization and OpenMP support. Run 1 was treated as a warm-up and discarded. The sequential baseline T_{seq} was calculated from Runs 2 and 3.

Run	Execution Time (s)	Use
Run 1	3.540000	Warm-up / discarded
Run 2	3.265000	Measured

Run 3	3.244000	Measured
-------	----------	----------

$$T_{seq} = (3.265000 + 3.244000) / 2 = 3.254500 \text{ s}$$

4. Multi-Thread Scaling and Empirical Amdahl Fitting

The OpenMP version was benchmarked at 1, 2, 4, and 8 threads. The processor exposes 8 logical processors, so a 16-thread hardware point was not used.

4.1 Analytical Derivation of Parallel Fraction (p)

For $k = 2$, the measured average runtime was $T_2 = 1.6815 \text{ s}$.

$$S_{emp}(2) = T_{seq} / T_2 = 3.2545 / 1.6815 = 1.9355$$

$$p = 2 \times [1 - (1 / S_{emp}(2))]$$

$$p = 2 \times [1 - (1 / 1.9355)] = 0.9667$$

Therefore, the empirically derived parallel fraction is approximately 0.9667, or 96.67%.

4.2 Empirical Measurement Log

Threads (k)	Run 1	Run 2	Run 3	Avg T k	S emp	S theo	Delta
1	3.535	3.839	3.329	3.5840	0.9081	1.0000	0.0919
2	1.673	1.696	1.667	1.6815	1.9355	1.9355	0.0000
4	1.154	1.033	1.195	1.1140	2.9215	3.6363	0.7149
8	0.639	0.651	0.636	0.6435	5.0575	6.4863	1.4288
16	N/A	N/A	N/A	N/A	N/A	N/A	N/A

The empirical speedup increased to approximately 5.06x at 8 logical threads. The theoretical Amdahl curve increasingly exceeded the measured curve as k increased, producing a larger reality gap at higher thread counts.

5. Hardware Penalty Experiment A: False Sharing

The experiment counted values in $[1, N]$ requiring more than 100 Collatz stopping steps. Two implementations were compared on 4 physical cores: a naive per-thread counter array and an OpenMP reduction.

Variant	Threads	Hits	Time (s)	Throughput (iter/s)	Penalty Ratio
Naive hit count[tid]++	4	10,491,962	1.070	12,174,767.11	1.2145
OpenMP reduction	4	10,491,962	0.881	14,786,605.43	1.0000 baseline

The naive counter-array implementation took about 1.2145 times the runtime of the reduction implementation in this measurement. Both variants produced the same hit count.

6. Hardware Penalty Experiment B: Loop Scheduling

Scheduling Clause	Chunk Size	Time (s)	Observed Behavior
schedule(static)	Default	1.008	Baseline static partitioning across threads.
schedule(static, 1000)	1,000	0.988	Slightly faster than default static in this run.
schedule(dynamic, 100)	100	0.936	Lowest measured time; fine-grained dynamic balancing.
schedule(dynamic, 10000)	10,000	0.945	Slightly slower than dynamic(100).
schedule(guided)	Exponential decay	0.965	Adaptive result between the dynamic and static measurements.

For this benchmark run, dynamic(100) produced the lowest measured execution time. Because these scheduling measurements were single observed runs, the differences should be reported as empirical observations rather than universal performance rankings.

7. Technical Analysis and Defense Questions

Q1. Micro-Architectural Root Cause of False Sharing

Variant 1 stores the per-thread hit counters next to each other in one integer array. With a 64-byte cache line, several counters can occupy the same cache line even though each thread updates a different array element. When different cores repeatedly write to those elements, the cache-coherence mechanism must transfer ownership and invalidate other cached copies of the same line. This creates cache-line ping-pong and additional coherence traffic. The effect is consistent with the measured result: the naive variant required 1.070 s, whereas the reduction variant required 0.881 s, giving a penalty ratio of 1.2145. OpenMP reduction avoids the repeatedly shared counter line by maintaining thread-private partial values and combining them after the parallel work.

Q2. Hyperthreading (SMT) Saturation and Physical Core Ceilings

The processor has 4 physical cores and 8 logical SMT threads. In the measured Collatz workload, speedup did not stop at the physical-core count: S_{emp} increased from approximately 2.92x at 4 threads to approximately 5.06x at 8 threads. Therefore, SMT provided additional throughput for this workload, but the scaling was not linear relative to the ideal 8x speedup. SMT threads on the same physical core share important execution resources, including core execution units, pipeline/front-end resources, and cache/memory bandwidth. Because the two logical threads are not two independent physical cores, resource contention limits ideal scaling.

Q3. Empirical vs. Theoretical Amdahl Discrepancy

The derived parallel fraction was $p = 0.9667$ (approximately 96.67%). Using this value, Amdahl's Law predicts increasingly high speedup as the thread count rises. The measured speedup, however, was lower than the theoretical value at higher k . At $k = 4$, S_{emp} was approximately 2.92 while S_{theo} was approximately 3.64; at $k = 8$, S_{emp} was approximately 5.06 while S_{theo} was approximately 6.49. Two physical factors omitted by the basic Amdahl equation are synchronization/thread-management overhead and contention for shared hardware resources such as cache and memory bandwidth. SMT resource sharing, cache-coherence traffic, scheduling overhead, and workload imbalance can further enlarge the reality gap.

Q4. The Scheduling Trade-Off Dilemma

In the measured scheduling experiment, `dynamic(100)` completed in 0.936 s and `dynamic(10000)` completed in 0.945 s, while default static scheduling required 1.008 s. The smaller dynamic chunk provided the best measured time, suggesting that improved load balancing outweighed the additional dynamic scheduling overhead for this run. The 10,000-iteration dynamic chunk reduced queue interaction but offered coarser load balancing and was slightly slower. In these measurements, there is no evidence that lock-queue contention at chunk size 100 outweighed the balancing benefit; `dynamic(100)` remained faster than `dynamic(10000)`. Therefore, a precise contention crossover chunk size cannot be established from the tested data alone.

8. Conclusion

The experiment demonstrated the Amdahl Reality Gap on the tested Intel Core i5-1155G7 system. The sequential baseline was 3.2545 s and the measured speedup reached approximately 5.06x at 8 logical threads. The derived parallel fraction was approximately 96.67%, but measured scaling remained below the Amdahl prediction as thread count increased. The false-sharing experiment showed a measurable penalty for adjacent per-thread counters, while the scheduling experiment showed that `dynamic(100)` produced the lowest observed runtime among the tested scheduling clauses. Overall, the results show that real parallel performance is affected by cache coherence, scheduling behavior, synchronization overhead, SMT resource sharing, and other hardware effects not represented in the idealized Amdahl model.

9. Submission Files

The final submission package should include the following separately generated artifacts: `hw_info.txt`, the C/C++ source code, `results.csv`, `speedup_plot.png`, and this written analysis/report. The measured values in this report were obtained from the user's actual benchmark runs.